# Informe PIC Lógica-Grafos

Alumnos: Isern Felipe Juan y Koprcina Sofia Lara

## Descripción del problema y decisiones de diseño:

El problema nos pide que construyamos un sistema con un modelo para estimar los tiempos de viajes con algoritmos de grafos para calcular las rutas sobre los tiempos estimados. Para eso, decidimos usar un modelo de grafo el cual cada esquina sea un nodo y cada calle sea una arista dirigida desde la esquina de la que sale a la esquina que llega. Decidimos que si una calle es avenida debería haber una arista saliendo de cada una de las esquinas y entrar a otras, esto técnicamente podría generar giros en U infinitos pero debido a que eso no tendría sentido y no pasa con Dijsktra ya que este algoritmo recuerda por qué vértices pasó no los repite

## Descripción del dataset utilizado (fuente o reglas de generación sintética, features elegidas y por qué).

Para el dataset, lo primero que hicimos fue usando OSMnx creamos un grafo dirigido directamente de un mapa real desde OpenStreetMap por lo que nos ahorra tener que pensar cómo crear el grafo y por encima, nos da un grafo de un mapa sobre una localidad real.

Luego hicimos un código para que cree valores aleatorios en cada arista y luego dependiendo la calle, le hicimos retoques al valor, como bajar el tiempo en caso de avenidas o subirlo en caso de que haya mal clima para que sea más realista y que no piense que tomar una cortada angosta va a tener la misma velocidad que una avenida.

## Metodología de entrenamiento del modelo: partición de datos, métricas usadas, resultados obtenidos, análisis de errores.

Dentro de la metodología de entrenamiento, para la parte de partición de datos más específicamente, decidimos que las features eran, dentro de los datos que nos pedían que tengamos, las siguientes:

- hora_dia, dia_semana, clima, es_feriado

y decidimos que de todos los datos que nos daba el ejercicio el target iba a tener que ser casi que obviamente:

- tiempo_real_viaje

Para el primer intento el dataset se dividió utilizando una estrategia de validación estándar, separando el 80% de los datos para entrenamiento y el 20% restante para testeo/evaluación. Esto nos dio los siguientes resultados.

- MAE (Error Absoluto Medio): 6.29 segundos
- RMSE (Raíz del Error Cuadrático Medio): 8.77 segundos
- $R^2$ (Coeficiente de Determinación): 0.2297

los cuales se nos hicieron malos, el MAE era muy alto y en viajes largos podría acumularse el error y dar tiempos muy diferentes, lo mismo para el RMSE y el $R^2$ tan bajo no era nada óptimo ya que estaba muy cerca de 0, así que intentamos bajarlo añadiendo la distancia a las features (una vez nos dimos cuenta sonaba muy lógico) y terminamos con resultados mucho más parecidos:

- MAE (Error Absoluto Medio): 2.66 segundos
- RMSE (Raíz del Error Cuadrático Medio): 4.55 segundos
- $R^2$ (Coeficiente de Determinación): 0.7842

Los cuales nos dejaron bastante más satisfechos y eran más cercanos a los que nos esperábamos

## Descripción del algoritmo de grafos implementado.

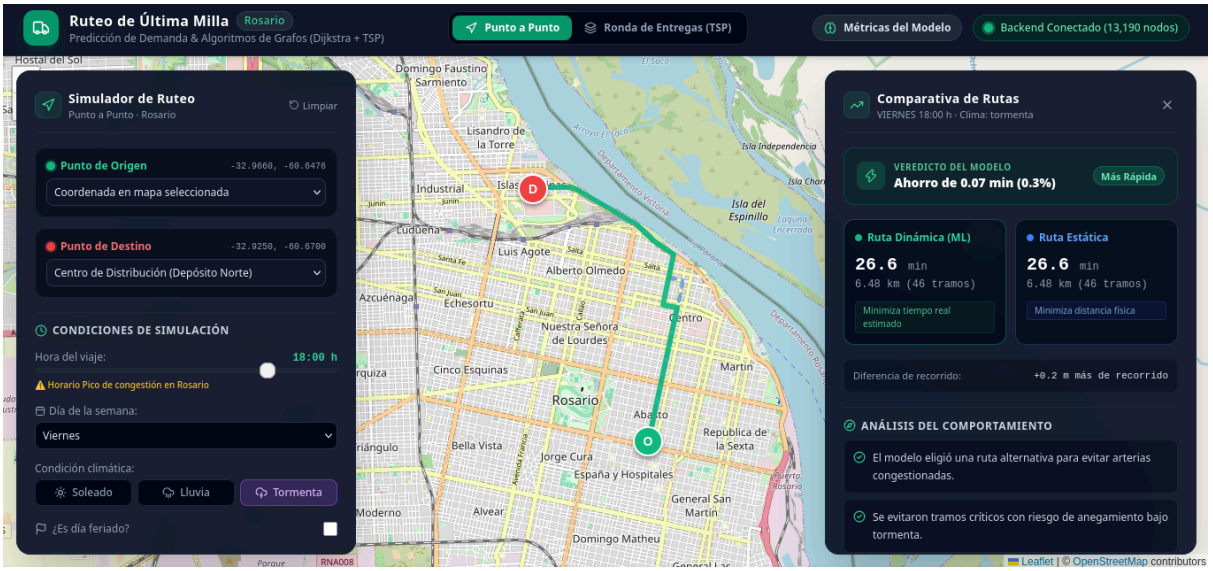
Para el ejercicio de buscar el camino óptimo de un punto A a un punto B dentro de el grafo que generamos en los apartados anteriores del trabajo, decidimos usar Dijkstra, como previamente habíamos intentado hacer otras cosas con grafos, teníamos ya conocida una librería para manipulación y otras cosas de grafos la cual ya tenía implementado en sí Dijkstra.

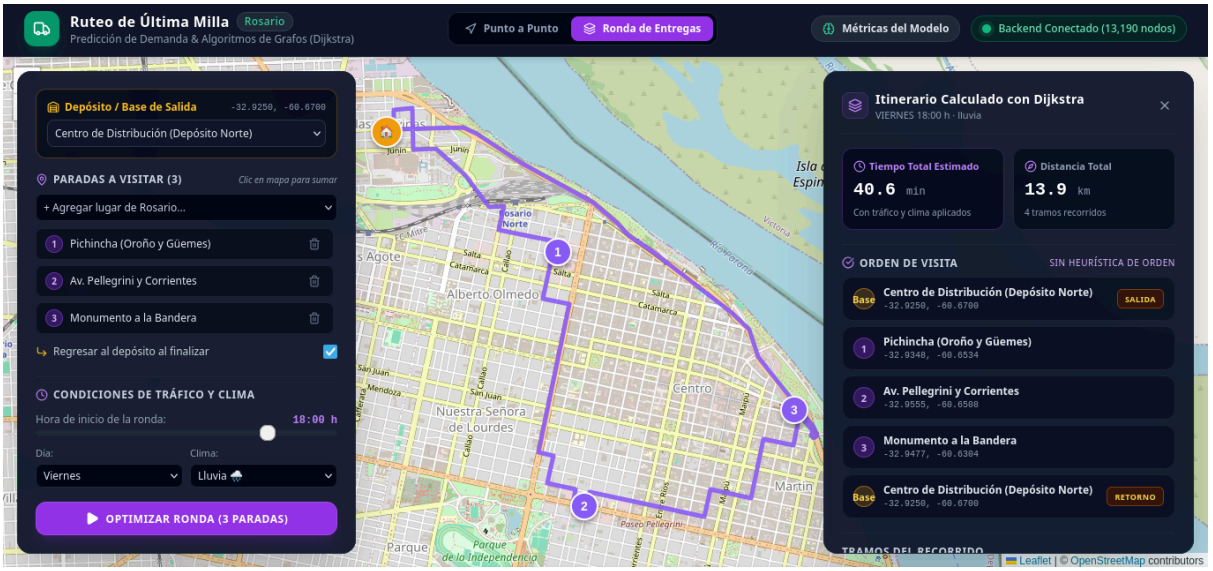
El algoritmo de Dijkstra el cual parte de un punto de partida con costo 0, empieza a explorar y acumular costos de las aristas que atraviesa hasta llegar al nodo el cual busca, esto garantiza que el camino que encuentre sea el más barato en términos de tiempo y por cómo funciona evita quedarse dando vueltas en bucle en las cuatro esquinas de una cuadra.

## Explicación de cómo se integran ambas partes (qué salida del modelo se usa como entrada del algoritmo de grafos, y por qué esa transformación es razonable).

Para la parte en la que combinamos los grafos y el modelo de regresión para estimar tiempos, pensamos en que cada calle tendría su costo dependiendo los diferentes factores (hora, clima, obras, etc.) y que dado un dataset con valores previos el modelo calculará los valores para las calles en el momento actual y el algoritmo de A* recorre los nodos para encontrar el mejor camino.

## Capturas de pantalla del frontend funcionando.




## Sección de uso de IA

El primer uso que le dimos a la IA fue encontrar un dataset sobre la ciudad de Rosario pero al momento de buscarlo, al no encontrarlo nos condujo a OSMnx para que podamos crear nuestro propio dataset con un grafo ya hecho y nos explico como hacer toda la implementación, cómo funcionaba el grafo que estábamos creando y cómo crear el dataset con las variables que le pedimos.

Luego le pedimos que nos ayude con la implementación de Machine learning, y cómo hacer para el mismo extraerlo de un archivo py para utilizarlo en otro dentro de la misma carpeta.

Tambien nos ayudo con usar correctamente la librería que teníamos para usar grafos y como estaba hecha la implementación de Dijkstra para poder usarla correctamente teniendo

en cuenta que cada nodo no tenía solo un valor guardado siendo este el peso si no tambien, tenía guardado muchas cosas extras como el clima la hora, etc.

Como aporte general nos ayudó mucho con errores de código cuando no compilaba o tiraba errores que no conocíamos.

En la realización de la página web se utilizó Antigravity, junto a los modelos de Gemini y Claude.

Primero, le dijimos a la IA que analizara el proyecto y sus necesidades gráficas para determinar qué herramientas eran las más indicadas para implementar en el back y frontend.

Para el backend concluyó que lo más óptimo era usar **Python** con **FastAPI**. En este proceso de análisis la IA se encontró con un problema, cuánto iba a tardar el proyecto en levantarse si este cada vez que lo quisiera hacer tenía que cargar el modelo y el grafo del mapa. Para esto, resolvió que la mejor solución era cargar el modelo de Random Forest de regresión y el grafo del mapa de Rosario en el backend (se encuentran en **backend/models** y **backend/src**) para ahorrar tiempo al levantar el proyecto.

Para el frontend concluyó en hacer una **Single Page Application**, usando **React**, **TypeScript**, **Vite** y **Tailwind CSS**.

El mapa es cargado con **Leaflet** y **OpenStreetMap**, el primero para mostrar los marcadores de origen y destino(s) y el segundo para obtener el mapa de Rosario y aprovecharse de su modelo topológico de nodos y vías para facilitar el cálculo del camino.

Un caso particular y bastante menor en el que se equivocó la IA fue en tener en cuenta al nodo de partida en “Ronda de Entregas” en el mapa del frontend como un nodo aparte; es decir, se lo contaba una vez al empezar el recorrido y otra vez más al finalizarlo, por lo que había un nodo de más en la cuenta, cuando en realidad era el mismo nodo contado dos veces.

Además, interpretó de agregar el uso de IA al frontend. No se le explicitó lo contrario a la hora de crearla, pero aún así se tuvo que borrar ya que este es el apartado de uso de IA.

A excepción de estos casos mínimos, no se presentaron otros errores en la realización de la página web. Podemos afirmar que fue un proceso muy fluido.

La IA ayudó en todo este trabajo, tanto en la generación del dataset y el entrenamiento del modelo como, y especialmente, en la parte de realización de la totalidad de la página web, ya que la integrante asignada a esta tarea tenía poco conocimiento de por dónde empezar y qué herramientas usar.

## Limitaciones conocidas y posibles extensiones.

**Limitaciones conocidas:**

Una gran limitación que tiene este proyecto es no poder escribir coordenadas específicas y estar fijo a puntos clave de la ciudad. Se pueden elegir coordenadas usando el mismo mapa, pero no se pueden escribir para elegir una dirección específica. Esto es muy difícil de

implementar ya que requiere una extensa geocodificación de cada coordenada posible del mapa, pero no es la magnitud que se busca en este proyecto actualmente.

**Posibles extensiones:**

Aquí se presentan algunas posibles extensiones al proyecto para volverlo más óptimo y de funcionamiento general.

- Guardado de rutas frecuentes o simulaciones.
- Geocodificación para escribir direcciones de Rosario.
- Animación del camión recorriendo la ruta seleccionada.
- Optimización de flotas completas con múltiples vehículos.
- Conexión con APIs de tráfico en tiempo real.